

FileStream Class in C# with Examples

In this article, I am going to discuss **FileStream Class in C#** with Examples. Please read our previous article where we discussed the basics of [File Handling in C#](#).

What is FileStream Class in C#?

The FileStream Class in C# provides a stream for file operations. It can be used to perform both Synchronous and Asynchronous Read and Write operations. With the help of FileStream Class, we can easily read and write data into files.

How to use FileStream Class in C#?

To use FileStream Class in C#, first of all, we need to include the **System.IO** namespace and then we need to create an instance of the FileStream class either to create a new file or to open an existing file. If you go to the definition of FileStream class, then you will see that there are many overloaded versions of Constructors available as shown in the below image.

```
public class FileStream : Stream
{
    ...public FileStream(string path, FileMode mode);
    ...public FileStream(SafeFileHandle handle, FileAccess access);
    ...public FileStream(IntPtr handle, FileAccess access);
    ...public FileStream(string path, FileMode mode, FileAccess access);
    ...public FileStream(SafeFileHandle handle, FileAccess access, int bufferSize);
    ...public FileStream(IntPtr handle, FileAccess access, bool ownsHandle);
    ...public FileStream(string path, FileMode mode, FileAccess access, FileShare share);
    ...public FileStream(SafeFileHandle handle, FileAccess access, int bufferSize, bool isAsync);
    ...public FileStream(IntPtr handle, FileAccess access, bool ownsHandle, int bufferSize);
    ...public FileStream(IntPtr handle, FileAccess access, bool ownsHandle, int bufferSize, bool isAsync);
    ...public FileStream(string path, FileMode mode, FileAccess access, FileShare share, int bufferSize);
    ...public FileStream(string path, FileMode mode, FileAccess access, FileShare share, int bufferSize, bool useAsync);
    ...public FileStream(string path, FileMode mode, FileAccess access, FileShare share, int bufferSize, FileOptions options);
    ...public FileStream(string path, FileMode mode, FileSystemRights rights, FileShare share, int bufferSize, FileOptions options);
    ...public FileStream(string path, FileMode mode, FileSecurity fileSecurity);
}
```

The simplest way to create an instance of FileStream Class is to use the following overloaded version of the Constructors.

public FileStream(string path, FileMode mode): This Constructor Initializes a new instance of the FileStream class with the specified path and creation mode.

Here,

1. **path:** A relative or absolute path for the file that the current FileStream object will encapsulate.
2. **mode:** A constant that determines how to open or create the file.

public FileStream(string path, FileMode mode, FileAccess access): This overloaded version initializes a new instance of the FileStream class with the specified path, creation mode, and read/write permission.

Here,

1. **path:** A relative or absolute path for the file that the current FileStream object will encapsulate.
2. **mode:** A constant that determines how to open or create the file.

3. **access**: A constant that determines how the file can be accessed by the FileStream object. This also determines the values returned by the System.IO.FileStream.CanRead and System.IO.FileStream.CanWrite properties of the FileStream object. System.IO.FileStream.CanSeek is true if the path specifies a disk file.

public FileStream(string path, FileMode mode, FileAccess access, FileShare share): This overloaded version Initializes a new instance of the System.IO.FileStream class with the specified path, creation mode, read/write permission, and sharing permission.

1. **path**: A relative or absolute path for the file that the current FileStream object will encapsulate.
2. **mode**: A constant that determines how to open or create the file.
3. **access**: A constant that determines how the file can be accessed by the FileStream object. This also determines the values returned by the System.IO.FileStream.CanRead and System.IO.FileStream.CanWrite properties of the FileStream object. System.IO.FileStream.CanSeek is true if the path specifies a disk file.
4. **share**: A constant that determines how the file will be shared by processes.

The **path** parameter is nothing but a string value that contains the path where the file is going to be created or the path of an existing file whose data we want to access. The rest three parameters i.e. **FileMode**, **FFileAccess** and **FileShare** are Enums and let us discuss these three Enums in detail.

FileMode in C#:

The FileMode specifies how the operating system should open a file. If you go to the definition of FileMode, then you will see that it is an Enum with the following structure.

```
namespace System.IO
{
    ...public enum FileMode
    {
        ...CreateNew = 1,
        ...Create = 2,
        ...Open = 3,
        ...OpenOrCreate = 4,
        ...Truncate = 5,
        ...Append = 6
    }
}
```

It has the following six constant values.

1. **CreateNew**: It specifies that the operating system should create a new file. This requires a System.Security.Permissions.FileIOPermissionAccess.Write permission. If the file already exists, a System.IO.IOException exception is thrown.

2. **Create:** It specifies that the operating system should create a new file like the CreateNew constant. But in this case, if the file already exists, it will be overwritten instead of throwing an Exception. This also requires System.Security.Permissions.FileIOPermissionAccess.Write permission. So, FileMode.Create is equivalent to requesting that if the file does not exist, use System.IO.FileMode.CreateNew; otherwise, use System.IO.FileMode.Truncate. If the file already exists but is a hidden file, then an UnauthorizedAccessException Exception is going to be thrown.
3. **Open:** It specifies that the operating system should open an existing file. The ability to open the file is dependent on the value specified by the System.IO.FileAccess Enumeration. A System.IO.FileNotFoundException exception is thrown if the file does not exist.
4. **OpenOrCreate:** It specifies that the operating system should open a file if it exists; otherwise, a new file should be created. If the file is opened with FileAccess.Read, System.Security.Permissions.FileIOPermissionAccess.Read permission is required. If the file access is FileAccess.Write, System.Security.Permissions.FileIOPermissionAccess.Write permission is required. If the file is opened with FileAccess.ReadWrite, both Systems.Security.Permissions.FileIOPermissionAccess.Read and System.Security.Permissions.FileIOPermissionAccess.Write permissions are required.
5. **Truncate:** It specifies that the operating system should open an existing file. When the file is opened, it should be truncated so that its size is zero bytes. This requires a System.Security.Permissions.FileIOPermissionAccess.Write permission. Attempts to read from a file opened with FileMode.Truncate causes a System.ArgumentException exception.
6. **Append:** It opens the file if it exists and then adds the content at the end of the file, or creates a new file. This requires a System.Security.Permissions.FileIOPermissionAccess.Append permission. FileMode.Append can be used only in conjunction with FileAccess.Write. Trying to seek a position before the end of the file throws a System.IO.IOException exception, and any attempt to read fails and throws a System.NotSupportedException exception.

FileAccess in C#:

It permits file for read, write, or read/write access. If you go to the definition of FileAccess, then you will see that it is an Enum with the following structure.

```
namespace System.IO
{
    ...public enum FileAccess
    {
        ...Read = 1,
        ...Write = 2,
        ...ReadWrite = 3
    }
}
```

It has the following three constant values.

1. **Read**— It gives read access to the file. Data can be read from the file. Combine with Write for read/write access.
2. **Write** – It gives Write access to the file. Data can be written to the file. Combine with Read for read/write access.

3. **ReadWrite** – It gives read and writes access to the file. Data can be written to and read from the file.

FileShare in C#:

It contains constants for controlling the kind of access other FileStream objects can have to the same file. For example, if one file is accessed by one FileStream object and a second FileStream object wants to access the file then what will happen? That means it opens a file with share permission. If you go to the definition of FileShare, you will see that it is an Enum with the following structure.

```
namespace System.IO
{
    ...public enum FileShare
    {
        ...None = 0,
        ...Read = 1,
        ...Write = 2,
        ...ReadWrite = 3,
        ...Delete = 4,
        ...Inheritable = 16
    }
}
```

It has the following six constant values.

1. **None**: Declines sharing of the current file. Any request to open the file (by this process or another process) will fail until the file is closed.
2. **Read**: Allows subsequent opening of the file for reading. If this flag is not specified, any request to open the file for reading (by this process or another process) will fail until the file is closed. However, even if this flag is specified, additional permissions might still be needed to access the file.
3. **Write**: Allows subsequent opening of the file for writing. If this flag is not specified, any request to open the file for writing (by this process or another process) will fail until the file is closed. However, even if this flag is specified, additional permissions might still be needed to access the file.
4. **ReadWrite**: Allows subsequent opening of the file for reading or writing. If this flag is not specified, any request to open the file for reading or writing (by this process or another process) will fail until the file is closed. However, even if this flag is specified, additional permissions might still be needed to access the file.
5. **Delete**: Allows subsequent deleting of a file.
6. **Inheritable**: Makes the file handle inheritable by child processes. This is not directly supported by Win32.

Example to Understand FileStream class in C#:

Let us understand how to perform Read and Write Operations on a File using FileStream Class in C# with Examples. In the below example, we will create a new file called “**MyFile.txt**” and saves it on the Disk. And then we will open this file, saves some text in it, and then close this file.

File Creation Example using FileStream Class in C#:

In the below example, we have done three things. First, we set the File Path where we want to create the File. Second, we create an instance of the FileStream class by specifying the File Path and File Mode as Create as we are going to create a New File. Finally, we have close the FileStream Object. In this example, the FileStream Object is going to create a new **MyFile.txt** file in the D drive. The following example code is self-explained, so please go through the comment lines for a better understanding.

```
using System;
using System.IO;
namespace FileHandlinDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Set the File Path where you want to Create the File in your Disk
            string FilePath = @"D:\MyFile.txt";
            //Create an Instance of FileStream Class By specifying the File Path and File Mode
            //FileMode is going to be Create as we are going to create a New File
            FileStream fileStream = new FileStream(FilePath, FileMode.Create);
            //Finally close the fileStream Object
            fileStream.Close();
            Console.WriteLine("File has been created and the Path is D:\\MyFile.txt");
            Console.ReadKey();
        }
    }
}
```

When you run the above code, you will get the following output.

```
File has been created and the Path is D:\MyFile.txt
```

File Open and Write Example using FileStream Class in C#:

The **MyFile.txt** file is created on the D drive. Now we will open this file and we will write some text in it. In the below example, first, we created an instance of FileStream class. Then Encoded a string into bytes and kept it into byte[] variable bytedata and finally using the Write() method of FileStream stored the string into the file. The following example code is self-explained, so please go through the comment lines for a better understanding.

```
using System;
using System.IO;
using System.Text;
namespace FileHandlinDemo
{
    class Program
    {
```

```

static void Main(string[] args)
{
    //Set the File Path where your File is Exist
    string FilePath = @"D:\MyFile.txt";
    //Create an Instance of FileStream Class By specifying the File Pa
Mode
    //FileMode is going to be Append as we are going to append some te
file
    FileStream fileStream = new FileStream(FilePath, FileMode.Append);
    //Create the byte array which contains the string data to be apper
File
    byte[] bytedata = Encoding.Default.GetBytes("C# Is an Object Orier
Programming Language");
    //Write the Byte Array into the File Stream Object using the Write
    //array (bytedata): The buffer containing data to write to the str
    //offset (0): The zero-based byte offset in the array from which t
copying bytes to the stream.
    //count (bytedata.Length): The maximum number of bytes to write.
    fileStream.Write(bytedata, 0, bytedata.Length);
    //Finally close the fileStream Object
    fileStream.Close();
    Console.WriteLine("Successfully saved file with data : C# Is an Ob
Oriented Programming Language");
    Console.ReadKey();
}
}
}

```

When you run the above code, you will get the following output.

Successfully saved file with data : C# Is an Object Oriented Programming Language

File Read Example using FileStream Class in C#:

We have already created one **MyFile.txt**, and we have also written some data into it. Now, we will see, how to read the data from the **MyFile.txt** file. In the below example, we have opened the file with the Read Permission and uses the StreamReader class to read the file and then print the data in the console. The following example code is self-explained, so please go through the comment lines for a better understanding.

```

using System;
using System.IO;
namespace FileHandlinDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Set the File Path where your File is Exist
            string FilePath = @"D:\MyFile.txt";

```

```

        //Create a string variable to hold the file data
        string data;
        //Create an Instance of FileStream Class By specifying the File Pa
Mode, FileAccess
        //FileMode is going to be Open as we are going to read the data fr
        //To Read the File, we are providing Read Access
        FileStream fileStream = new FileStream(filePath, FileMode.Open,
FileAccess.Read);
        //Create an Instance of StreamReader Object to Read the Data from
        //To the Constructor of StreamReader, pass the fileStream Object i
stream to be read.
        using (StreamReader streamReader = new StreamReader(fileStream))
        {
            //ReadToEnd method reads all characters from the current posit
end of the stream.
            //It will return the rest of the stream as a string, from the
position to the end.
            //If the current position is at the end of the stream, returns
string ("").
            data = streamReader.ReadToEnd();
        }
        //Finally Print the data in the Console
        Console.WriteLine(data);
        Console.ReadKey();
    }
}
}

```

When you run the above code, you will get the following output.

C# Is an Object Oriented Programming Language

Note: The FileStream class in C# is used for reading and writing files. It is part of the **System.IO** namespace. To manipulate files using FileStream, you need to create an object of FileStream class. This object has four parameters; the Name of the File, FileMode, FileAccess, and FileShare.

In the next article, I am going to discuss [StreamReader and StreamWriter Class in C#](#) with Examples. Here, in this article, I try to explain **FileStream Class in C#** with Examples. I hope you enjoy this FileStream Class in C# with Examples article. I would like to have your feedback. Please post your feedback, question, or comments about this article.